ECS 408/608 : Operating System

Assignment 2 : Operations on Process

Name: Rudra Baunk **Roll No.:** 22268

Problem 1: Parent-Child Process Creation

Objective: To create a parent and child process using `fork()` and display their respective PIDs.

Code

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {

    printf("Name: Rudra Baunk \n");
    printf("------------------------------\n");

    pid_t pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        exit(1);
    }

    if (pid == 0) {
        printf("Hi! I am a child process.\n");
        printf("My PID is %d.\n", getpid());
        printf("MY Parent PID is %d.\n", getppid());
    }
    else {
        printf("Hi! I am a parent process.\n");
        printf("My PID is %d.\n", getpid());
        printf("My Child PID is %d.\n", pid);
        wait(NULL);
    }

    return 0;
}
```

Output

```
rudrabaunk@LAPTOP-01SLCLOU:/mnt/c/D_drive/OS/assignment 2$ echo "Name= Rudra Baunk | Rollnumber : 22268"
Name= Rudra Baunk | Rollnumber : 22268
rudrabaunk@LAPTOP-01SLCLOU:/mnt/c/D_drive/OS/assignment 2$ gcc q1.c -o q1
rudrabaunk@LAPTOP-01SLCLOU:/mnt/c/D_drive/OS/assignment 2$ ./q1
Name: Rudra Baunk
------------------------------
Hi! I am a parent process.
My PID is 514.
My Child PID is 515.
Hi! I am a child process.
My PID is 515.
MY Parent PID is 514.
rudrabaunk@LAPTOP-01SLCLOU:/mnt/c/D_drive/OS/assignment 2$
```

Problem 2: Merge Sort (Parent) and Quick Sort (Child)

Objective: To demonstrate independent execution of parent and child processes where: Parent performs Merge Sort and Child performs Quick Sort.

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

void merge(int arr[], int l, int m, int r) {
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;
    int L[n1], R[n2];
    for (i = 0; i < n1; i++) L[i] = arr[l + i];
    for (j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
    i = 0; j = 0; k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}
void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
```

```
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {

    printf("Name: Rudra Baunk\n");
    int n;
    printf("Enter number of integers: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d integers: ", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    pid_t pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        exit(1);
    }
    if (pid == 0) {
        printf("Hi! I am a child process.\n");
        printf("My PID is %d.\n", getpid());
        printf("MY Parent PID is %d.\n", getppid());

        quickSort(arr, 0, n - 1);

        printf("Quick Sort Result: ");
        printArray(arr, n);
        exit(0);
    }
    else {
        wait(NULL);
        printf("Hi! I am a parent process.\n");
        printf("My PID is %d.\n", getpid());
        mergeSort(arr, 0, n - 1);
        printf("Merge Sort Result: ");
        printArray(arr, n);
```

```
    }
    return 0;
}
```

Output

```
rudrabaunk@LAPTOP-O1SLCLOU:/mnt/c/D_drive/OS/assignment 2$ echo "Name= Rudra Baunk | Rollnumber : 22268"
Name= Rudra Baunk | Rollnumber : 22268
rudrabaunk@LAPTOP-O1SLCLOU:/mnt/c/D_drive/OS/assignment 2$ gcc q2.c -o q2
rudrabaunk@LAPTOP-O1SLCLOU:/mnt/c/D_drive/OS/assignment 2$ ./q2
Name: Rudra Baunk
Enter number of integers: 5
Enter 5 integers: 1
2
4
5
6
Hi! I am a child process.
My PID is 549.
MY Parent PID is 548.
Quick Sort Result: 1 2 4 5 6
Hi! I am a parent process.
My PID is 548.
Merge Sort Result: 1 2 4 5 6
rudrabaunk@LAPTOP-O1SLCLOU:/mnt/c/D_drive/OS/assignment 2$ |
```

Problem 3: Orphan Process Creation

Objective: To create an orphan process by terminating the parent before the child. Parent calculates factorial and child generates Fibonacci sequence.

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
long long factorial(int x) {
    long long fact = 1;
    for (int i = 1; i <= x; i++)
        fact *= i;
    return fact;
}
void fibonacci(int m) {
    int a = 0, b = 1, c;
    printf("Fibonacci numbers for %d numbers: ", m);
    for (int i = 0; i < m; i++) {
        printf("%d ", a);
        c = a + b;
        a = b;
        b = c;
    }
    printf("\n");
}

int main() {
    printf("Name: Rudra Baunk\n");
    int n, m;
    printf("Enter n for factorial: ");
```

```
    scanf("%d", &n);
    printf("Enter m for fibonacci (m >> n): ");
    scanf("%d", &m);
    pid_t pid = fork();
    if (pid < 0) {
        perror("Fork failed");
        exit(1);
    }

    if (pid > 0) {
        printf("Hi! I am a parent process.\n");
        printf("My PID is %d.\n", getpid());
        printf("Factorial for number %d: %lld.\n", n, factorial(n));
        printf("I am terminated earlier than my child.\n");
        exit(0);
    }
    else {
        sleep(2);
        printf("Hi! I am a child process.\n");
        printf("My PID is %d.\n", getpid());
        printf("MY Parent PID is %d.\n", getppid());

        fibonacci(m);
    }
    return 0;
}
```

Output

```
rudrabaunk@LAPTOP-O1SLCLOU:/mnt/c/D_drive/OS/assignment 2$ echo "Name= Rudra Baunk | Rollnumber : 22268"
Name= Rudra Baunk | Rollnumber : 22268
rudrabaunk@LAPTOP-O1SLCLOU:/mnt/c/D_drive/OS/assignment 2$ gcc q3.c -o q3
rudrabaunk@LAPTOP-O1SLCLOU:/mnt/c/D_drive/OS/assignment 2$ ./q3
Name: Rudra Baunk
Enter n for factorial: 6
Enter m for fibonacci (m >> n): 8
Hi! I am a parent process.
My PID is 578.
Factorial for number 6: 720.
I am terminated earlier than my child.
rudrabaunk@LAPTOP-O1SLCLOU:/mnt/c/D_drive/OS/assignment 2$ Hi! I am a child process.
My PID is 579.
MY Parent PID is 306.
Fibonacci numbers for 8 numbers: 0 1 1 2 3 5 8 13
```

Problem 4: Process Tree Creation

Objective: To create a process tree structure using multiple `fork()` calls.

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    printf("Name: Rudra Baunk\n");
```

```
    pid_t Q, R, S, T, U;
    printf("Hi! I am a root process P.\n");
    printf("My PID is %d.\n", getpid());
    Q = fork();
    if (Q < 0) {
        perror("Fork failed");
        exit(1);
    }
    if (Q == 0) {
        printf("Hi! I am a child process Q of Parent P.\n");
        printf("My PID is %d.\n", getpid());
        printf("MY Parent PID is %d.\n", getppid());
        S = fork();
        if (S == 0) {
            printf("Hi! I am a child process S of Parent Q.\n");
            printf("My PID is %d.\n", getpid());
            printf("MY Parent PID is %d.\n", getppid());
            exit(0);
        }
        T = fork();
        if (T == 0) {
            printf("Hi! I am a child process T of Parent Q.\n");
            printf("My PID is %d.\n", getpid());
            printf("MY Parent PID is %d.\n", getppid());
            exit(0);
        }
        wait(NULL);
        wait(NULL);
        exit(0);
    }
    else {

        R = fork();

        if (R == 0) {
            printf("Hi! I am a child process R of Parent P.\n");
            printf("My PID is %d.\n", getpid());
            printf("MY Parent PID is %d.\n", getppid());
            U = fork();
            if (U == 0) {
                printf("Hi! I am a child process U of Parent R.\n");
                printf("My PID is %d.\n", getpid());
                printf("MY Parent PID is %d.\n", getppid());
                exit(0);
            }
            wait(NULL);
            exit(0);
        }
        wait(NULL);
        wait(NULL);
    }
    return 0;
}
```

Output

```
rudrabaunk@LAPTOP-O1SLCLOU:/mnt/c/D_drive/OS/assignment 2$ echo "Name= Rudra Baunk | Rollnumber : 22268"
Name= Rudra Baunk | Rollnumber : 22268
rudrabaunk@LAPTOP-O1SLCLOU:/mnt/c/D_drive/OS/assignment 2$ gcc q4.c -o q4
rudrabaunk@LAPTOP-O1SLCLOU:/mnt/c/D_drive/OS/assignment 2$ ./q4
Name: Rudra Baunk
Hi! I am a root process P.
My PID is 595.
Hi! I am a child process Q of Parent P.
My PID is 596.
MY Parent PID is 595.
Hi! I am a child process R of Parent P.
My PID is 597.
MY Parent PID is 595.
Hi! I am a child process S of Parent Q.
My PID is 598.
MY Parent PID is 596.
Hi! I am a child process T of Parent Q.
Hi! I am a child process U of Parent R.
My PID is 600.
My PID is 599.
MY Parent PID is 596.
MY Parent PID is 597.
```